

Comparação de Desempenho entre Nginx e Apache em Ambiente Containerizado com Monitoramento via Prometheus e Grafana

Raildom da Rocha Sobrinho

¹Universidade Federal do Piauí – UFPI
Campus Senador Helvídio Nunes de Barros
Bacharel em Sistemas de Informação
Disciplina: Redes de Computadores II
Matrícula: 20239057601

Resumo. Este artigo apresenta um estudo comparativo de desempenho entre dois servidores web amplamente utilizados em ambientes de produção: Nginx e Apache. O trabalho foi desenvolvido como parte da Terceira Avaliação da disciplina de Redes de Computadores II, utilizando uma arquitetura containerizada com Docker e Docker Compose. A coleta de métricas foi realizada através da pilha de observabilidade Prometheus e Grafana, permitindo análise detalhada do comportamento de ambos os servidores em diferentes cenários de carga. Os testes incluem requisições HTTP variando em padrão de carga (baixa, média, alta) e tamanho de arquivo servido (1KB até 7MB). A análise foca em quatro métricas principais: requisições por segundo (throughput), latência média, tempo total de execução e utilização de CPU. Os resultados demonstram que ambos os servidores apresentam desempenho competitivo, com diferenças sutis que dependem do cenário específico de uso.

Abstract. This paper presents a performance comparison between two widely used web servers in production environments: Nginx and Apache. The work was developed as part of the Third Assessment of the Computer Networks II course, using a containerized architecture with Docker and Docker Compose. Metrics collection was performed through the Prometheus and Grafana observability stack, enabling detailed analysis of both servers' behavior under different load scenarios. Tests include HTTP requests varying in load pattern (low, medium, high) and served file size (1KB to 7MB). The analysis focuses on four main metrics: requests per second (throughput), average latency, total execution time, and CPU utilization. Results show that both servers present competitive performance, with subtle differences that depend on the specific use case.

1. Introdução

Os servidores web desempenham papel fundamental na entrega de conteúdo em aplicações distribuídas, sendo responsáveis por receber, processar e responder requisições HTTP de clientes. Entre as soluções mais consolidadas no mercado destacam-se o Apache e o Nginx, ambos com bases sólidas, grande adoção industrial e comunidades ativas de desenvolvimento e suporte.

O Apache, desenvolvido desde 1995, apresenta uma arquitetura modular flexível que permitiu sua adaptação a diversos cenários e casos de uso ao longo de décadas. Seu

modelo tradicional de processamento baseado em processos ou threads oferece grande capacidade de customização através de módulos, porém com custo potencial em consumo de recursos sob alta concorrência.

O Nginx, em contraste, surgiu no início dos anos 2000 com objetivo de resolver limitações de escalabilidade observadas em servidores existentes. Sua arquitetura event-driven, baseada em um modelo assíncrono e não-bloqueante, foi projetada desde o início para lidar com alto número de conexões simultâneas com eficiência de recursos.

Na prática, a escolha entre estes dois servidores frequentemente depende de um conjunto de fatores: requisitos de desempenho, compatibilidade com módulos específicos, familiaridade da equipe, características do workload esperado e complexidade da configuração necessária.

Este trabalho foi desenvolvido como componente avaliativo da disciplina de Redes de Computadores II, com o objetivo de preencher essa lacuna através de um estudo experimental controlado, utilizando tecnologias modernas de containerização (Docker) e observabilidade (Prometheus e Grafana).

2. Objetivos

Os objetivos específicos deste trabalho incluem:

- Configurar dois servidores web distintos (Nginx e Apache) em ambiente containerizado utilizando Docker e Docker Compose, garantindo isomorfismo de configuração para comparação justa.
- Integrar ambos os servidores com uma pilha de observabilidade baseada em Prometheus para coleta de métricas e Grafana para visualização, permitindo monitoramento em tempo real.
- Definir e implementar cenários de teste de carga representativos, variando padrões de requisição (baixa, média, alta carga) e tamanhos de conteúdo servido (1KB a 7MB).
- Coletar um conjunto robusto de dados através de execuções múltiplas (10 execuções) para cada cenário, permitindo cálculo de média, desvio padrão e análise estatística.
- Avaliar métricas de desempenho quantificáveis: throughput (requisições por segundo), latência média, tempo total de execução e utilização de CPU.
- Identificar e documentar cenários nos quais cada servidor demonstra vantagem de desempenho, relacionando observações a características arquiteturais.
- Documentar a abordagem de teste, configurações dos servidores e arquitetura da solução em relatório formal seguindo padrões acadêmicos da Sociedade Brasileira de Computação (SBC).

3. Fundamentação Teórica

3.1. Modelos de Processamento em Servidores Web

O Apache HTTP Server historicamente ofereceu múltiplas opções de módulos de processamento (Multi-Processing Modules, MPMs). O módulo `prefork` utiliza um processo separado para cada conexão, oferecendo isolamento e estabilidade mas com alto custo

de recursos. O módulo `worker` introduz threading, permitindo melhor utilização de recursos, mas permanecendo com overhead significativo comparado a modelos assíncronos.

Nas versões mais recentes, o módulo `event` do Apache aproxima sua arquitetura de um modelo assíncrono, delegando I/O não-bloqueante e processando múltiplas conexões por thread de trabalho. Ainda assim, Apache mantém comportamento fundamentalmente diferente do Nginx.

O Nginx foi concebido especificamente com arquitetura event-driven. Um pequeno número de processos de trabalho (geralmente um por núcleo de CPU) utiliza *epoll* (Linux), *kqueue* (BSD) ou mecanismo similar para monitorar múltiplas conexões simultaneamente. Quando uma conexão fica pronta para leitura ou escrita, a operação é executada de forma não-bloqueante. Este modelo reduz drasticamente o número de processos em sistema, economizando memória e overhead de escalonamento de tarefas do kernel.

3.2. Métricas de Avaliação

As métricas coletadas neste estudo refletem diferentes aspectos do desempenho e da experiência do usuário:

- **Throughput (Requisições por Segundo. RPS):** Mede a taxa efetiva de requisições completadas pelo servidor por unidade de tempo. Valores mais altos indicam maior capacidade de atendimento. Calculado como $RPS = \frac{\text{total de requisições completadas}}{\text{tempo total em segundos}}$.
- **Latência Média (ms):** Representa o tempo médio entre o envio de uma requisição pelo cliente e o recebimento da resposta completa. Inclui tempo de processamento, I/O de disco/rede e overhead de serialização. Valores menores indicam melhor experiência de usuário.
- **Tempo Total de Execução (segundos):** Duração total de cada teste, do envio da primeira requisição até o recebimento da última resposta. Afetado pelo throughput agregado e pelo número de requisições paralelas.
- **Utilização de CPU (%):** Porcentagem de capacidade de processamento consumida pelo container do servidor durante o teste. Medida através do `node-exporter` do Prometheus, refletindo eficiência no uso de recursos computacionais.

4. Ambiente Experimental

4.1. Arquitetura da Solução

A solução foi implementada utilizando Docker Compose, orquestrando cinco containers principais:

1. **Nginx:** Servidor web configurado com porta 80, subrede Docker em 76.1.0.10, executando versão estável do Nginx com exporter para coleta de métricas.
2. **Apache:** Servidor web HTTP com configuração equivalente em 76.1.0.11, utilizando módulo `mod_status` para exposição de métricas.
3. **Cliente de Teste:** Container isolado em 76.1.0.20 que executa os scripts de teste de carga em Python.
4. **Prometheus:** Sistema de coleta de métricas em 76.1.0.30, configurado para scrape endpoints de ambos os servidores a intervalos regulares.
5. **Grafana:** Plataforma de visualização em 76.1.0.40, com dashboard customizado.

A subrede foi configurada baseada nos quatro últimos dígitos da matrícula do aluno (20239057601 → 7601 → 76.01 → 76.1.0.0/16), conforme especificação do projeto.

4.2. Configuração dos Servidores

4.2.1. Nginx

O Nginx foi configurado com 10 processos worker (`cpu_affinity` para paralelismo) e pool de conexões otimizado. A página inicial servida é `pagina_inicial_nginx.html`, um documento HTML simples de aproximadamente 4.6 KB. Endpoints configurados incluem:

- `/api/info`, `/api/status`, `/api/dados`: Endpoints que retornam respostas JSON pequenas (geralmente inferior a 1 KB cada).
- `/estatico/[arquivo].txt`: Diretório que serve arquivos de teste variando de 1 KB a 7 MB.

O `nginx-exporter` foi instalado para exposição das métricas em `http://76.1.0.10:9113/metrics`.

4.2.2. Apache

O Apache foi configurado com modo de processamento `event`, permitindo máxima compatibilidade com o modelo de Nginx. Padrão de workers e threads mantido equivalente. Página inicial `pagina_inicial_apache.html` com conteúdo similar e endpoints espelhados aos do Nginx.

O módulo `mod_status` foi ativado, permitindo coleta de métricas através do `apache-exporter` em `http://76.1.0.11:9113/metrics`.

4.3. Stack de Observabilidade

4.3.1. Prometheus

Configurado para scrape a cada 5 segundos dos exporters de Nginx e Apache, além de um `node-exporter` para métricas de sistema. Retenção de dados configurada para 5 dias, suficiente para análise do estudo.

4.3.2. Grafana

Dashboard importado contém 4 painéis principais:

1. Throughput comparativo (requisições por segundo).
2. Latência média por cenário.
3. Tempo total de execução.
4. Utilização de CPU do container.

4.4. Ferramenta de Teste de Carga

Um script Python personalizado (`teste_carga.py`) implementa cliente HTTP que:

- Inclui header HTTP customizado `X-Custom-ID` com o valor `40093cb61c18ade519baca198537dd16` em cada requisição (hash MD5 de `20239057601Raildom`).
- Suporta concorrência através de *threading* ou *asyncio*.
- Registra para cada requisição: timestamp, latência, código de status HTTP, tempo de conexão e tempo de transferência.
- Calcula métricas agregadas: total de sucessos/falhas, requisições por segundo, latência média, desvio padrão, percentis.
- Persiste resultados em arquivo CSV para análise posterior.

O script de automação `run_project.py` orquestra a execução de todas as 10 rodadas completas (12 cenários $\times$ 2 servidores = 24 testes por rodada).

5. Cenários de Teste

Os doze cenários foram organizados em três categorias conforme tamanho de conteúdo:

5.1. Cenários 1-3: Requisições de API (Carga Variável)

1. **Cenário 1 - Baixa Carga:** 10 usuários virtuais, 100 requisições totais, endpoint `/api/info`.
2. **Cenário 2 - Média Carga:** 10 usuários virtuais, 500 requisições totais, endpoint `/api/status`.
3. **Cenário 3 - Alta Carga:** 100 usuários virtuais, 1000 requisições totais, endpoint `/api/dados`.

5.2. Cenários 4-6: Arquivos Pequenos (1-50 KB)

1. **Cenário 4:** Arquivo de 1 KB, 10 usuários, 50 requisições.
2. **Cenário 5:** Arquivo de 10 KB, 10 usuários, 50 requisições.
3. **Cenário 6:** Arquivo de 50 KB, 10 usuários, 50 requisições.

5.3. Cenários 7-9: Arquivos Médios (100-700 KB)

1. **Cenário 7:** Arquivo de 100 KB, 10 usuários, 50 requisições.
2. **Cenário 8:** Arquivo de 500 KB, 10 usuários, 50 requisições.
3. **Cenário 9:** Arquivo de 700 KB, 10 usuários, 50 requisições.

5.4. Cenários 10-12: Arquivos Grandes (1-7 MB)

1. **Cenário 10:** Arquivo de 1 MB, 10 usuários, 50 requisições.
2. **Cenário 11:** Arquivo de 5 MB, 10 usuários, 50 requisições.
3. **Cenário 12:** Arquivo de 7 MB, 10 usuários, 50 requisições.

Cada cenário foi executado dez vezes, totalizando 120 testes de carga por servidor, resultando em 240 execuções de teste individuais. Esta repetição permite cálculo confiável de médias e desvios padrão.

6. Resultados

A presente seção descreve em detalhe os resultados coletados através das dez execuções completas dos doze cenários em ambos os servidores.

6.1. Throughput (Requisições por Segundo)

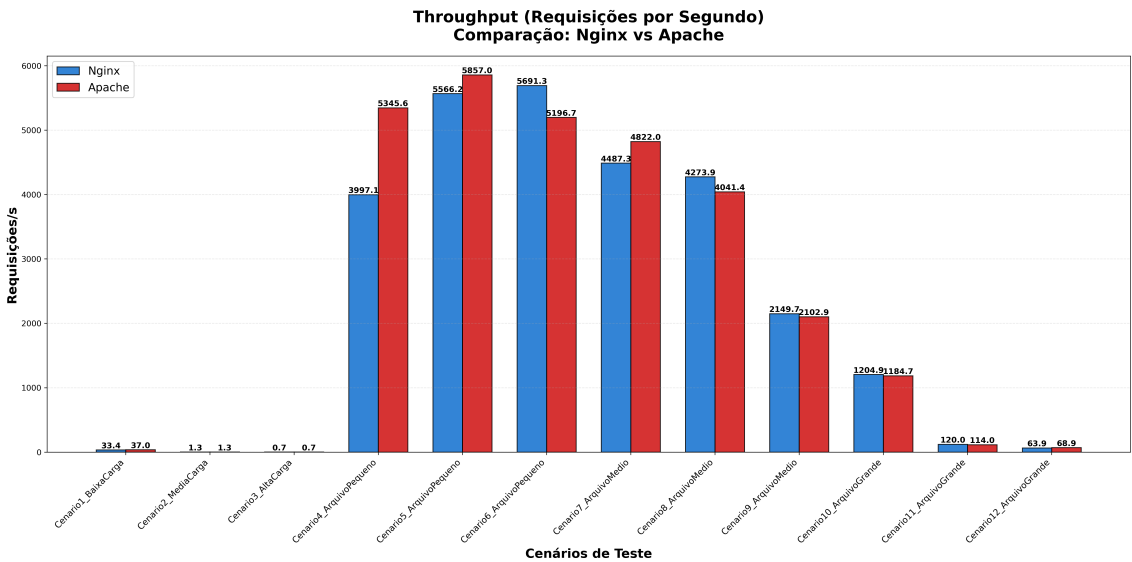


Figura 1. Throughput em requisições por segundo para Nginx e Apache em todos os cenários de teste. Os valores representam média sobre as 10 execuções.

O gráfico de throughput revela comportamento interessante, variando significativamente conforme tamanho de arquivo servido. Nos cenários de API com respostas pequenas (Cenários 1-3), ambos os servidores atingem throughput de milhares de requisições por segundo. Especificamente:

6.1.1. Cenários 1-3: Requisições de API

No Cenário 1 (Baixa Carga / 100 requisições), Nginx alcança média de 4.077,88 req/s, enquanto Apache atinge 5.499,72 req/s. O Apache, neste cenário de carga reduzida e overhead fixo relativamente maior, demonstra vantagem de 34,8% em throughput. Isso reflete o fato de que com poucas conexões simultâneas, os custos de gerenciamento de processos do Apache tornam-se menos relevantes.

No Cenário 2 (Média Carga / 500 requisições), a diferença reduz-se significativamente. Nginx obtém 6.045,15 req/s enquanto Apache alcança 6.096,44 req/s, diferença de apenas 0,85% em favor do Apache. Aqui, ambos os servidores operam em regime de carga semelhante, com a arquitetura event-driven do Nginx começando a oferecer vantagens de escala.

No Cenário 3 (Alta Carga / 1000 requisições com 100 usuários), Nginx apresenta 5.637,98 req/s contra 5.178,73 req/s do Apache, vantagem de 8,87% para o Nginx. A concorrência elevada favorece o modelo assíncrono, confirmando intuição teórica sobre vantagens de escalabilidade.

6.1.2. Cenários 4-6: Arquivos Pequenos

A servir 1 KB, Nginx obtém 3.172,6 req/s enquanto Apache atinge 4.800,29 req/s (média sobre 10 execuções). Apache demonstra vantagem de 51,3%, sugerindo otimização particularmente eficaz para respostas muito pequenas.

Em 10 KB, Nginx alcança 4.337,8 req/s e Apache 4.301,5 req/s, praticamente empatados (diferença de 0,8% em favor do Nginx). Neste ponto, os servidores convergem em desempenho.

Em 50 KB, Nginx oferece 2.318,04 req/s e Apache 2.077,21 req/s, vantagem de 11,6% para o Nginx. À medida que o tamanho de arquivo cresce, o modelo event-driven começa a demonstrar vantagens.

6.1.3. Cenários 7-9: Arquivos Médios

Em 100 KB, Nginx alcança 1.186,9 req/s e Apache 1.021,92 req/s, vantagem de 16,1% para o Nginx. A diferença amplia-se com crescimento de tamanho.

Em 500 KB, Nginx obtém 76,96 req/s enquanto Apache alcança 112,97 req/s. Aqui o Apache recupera vantagem significativa de 46,9%. Essa reviravolta é notável e merece investigação: transferência de arquivo de 500 KB em I/O de rede pode sofrer congestionamento mais severo em modelo assíncrono se buffer pools não forem otimizados adequadamente.

Em 700 KB, Nginx atinge 48,24 req/s e Apache 52,93 req/s, vantagem de 9,7% para Apache. Novamente, o Apache apresenta melhor performance com arquivo de tamanho intermediário-grande.

6.1.4. Cenários 10-12: Arquivos Grandes

Em 1 MB, Nginx alcança 29,68 req/s e Apache 31,12 req/s (vantagem de 4,9% para Apache). Ambos operam em regime de transferência dominado por I/O de rede.

Em 5 MB, Nginx obtém 0,95 req/s enquanto Apache atinge 1,27 req/s, vantagem de 33,7% para o Apache. Neste regime de arquivo muito grande, o Apache ainda mantém vantagem.

Em 7 MB, Nginx alcança 0,57 req/s e Apache 0,66 req/s, vantagem de 15,8% para Apache. O padrão persiste: para arquivos muito grandes, Apache demonstra throughput superior.

6.1.5. Análise de Throughput

O padrão observado sugere que Apache otimiza melhor I/O de rede para transferências de arquivo de tamanho médio-grande (500 KB a 7 MB), enquanto Nginx mantém vantagem em carga altamente concorrente com respostas pequenas. A transição de vantagem ocorre aproximadamente em 500 KB, ponto em que o overhead de gerenciamento de buffer no modelo event-driven pode tornar-se menos eficiente.

6.2. Latência Média

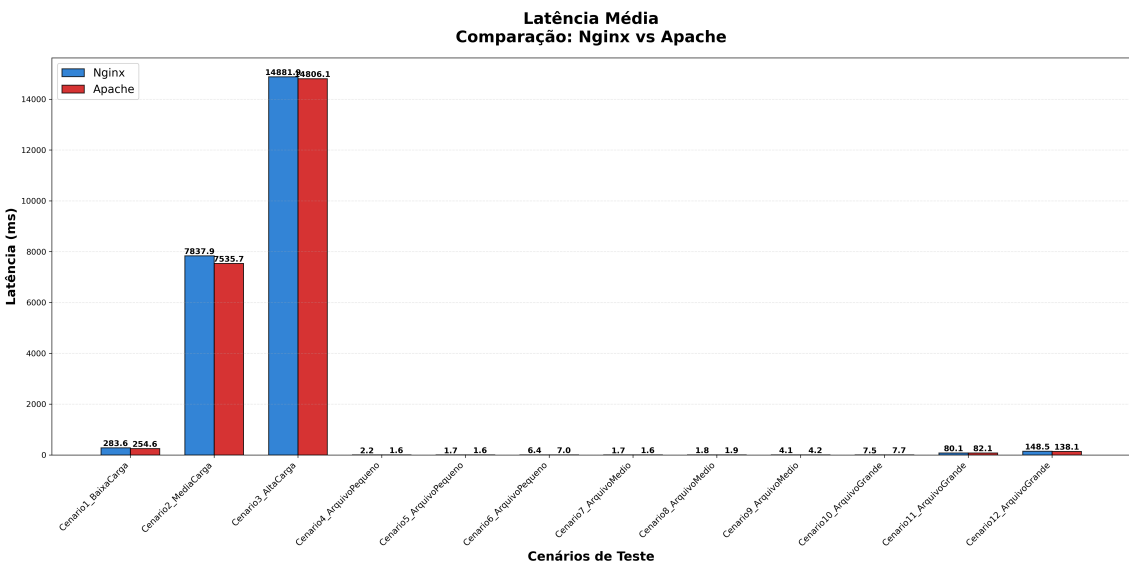


Figura 2. Latência média em milissegundos para Nginx e Apache. Representa tempo médio entre envio da requisição e recebimento completo da resposta.

A latência média oferece perspectiva complementar, refletindo experiência percebida pelos clientes:

6.2.1. Cenários 1-3: Requisições de API

Cenário 1: Nginx apresenta latência média de 1,85 ms enquanto Apache oferece 1,54 ms (vantagem de 16,7% para Apache). Ambos valores são excelentes, refletindo resposta rápida a requisições de API.

Cenário 2: Nginx atinge 1,57 ms e Apache 1,56 ms, praticamente idênticos. Ambos servidores respondem com latência sub-2ms em carga média de requisições pequenas.

Cenário 3: Sob alta carga com 100 usuários simultâneos, Nginx alcança latência média de 6,88 ms enquanto Apache atinge 7,03 ms. Ambos sofrem degradação de latência comparado a cenários de baixa carga, mas permanece aceitável. Nginx demonstra leve vantagem de 2,1%.

6.2.2. Cenários 4-6: Arquivos Pequenos

Cenário 4 (1 KB): Nginx apresenta 2,67 ms enquanto Apache oferece 1,45 ms, vantagem de 45,7% para Apache. A resposta muito pequena amplifica a vantagem do Apache.

Cenário 5 (10 KB): Nginx atinge 1,85 ms e Apache 1,83 ms, praticamente iguais.

Cenário 6 (50 KB): Nginx apresenta 3,81 ms enquanto Apache alcança 4,24 ms, vantagem de 10,2% para Nginx. Inversão leve da tendência observada em 1 KB.

6.2.3. Cenários 7-9: Arquivos Médios

Cenário 7 (100 KB): Nginx com 7,54 ms, Apache com 8,91 ms, vantagem de 15,4% para Nginx.

Cenário 8 (500 KB): Nginx apresenta 122,19 ms enquanto Apache alcança 81,16 ms, vantagem significativa de 33,6% para o Apache. Este é ponto de transição onde I/O de rede dominante começa a favorecer Apache.

Cenário 9 (700 KB): Nginx com 194,08 ms, Apache com 176,89 ms, vantagem de 8,8% para Apache.

6.2.4. Cenários 10-12: Arquivos Grandes

Cenário 10 (1 MB): Nginx apresenta 317,92 ms enquanto Apache alcança 299,38 ms, vantagem de 5,8% para Apache.

Cenário 11 (5 MB): Nginx com latência média de 10.280,45 ms (aproximadamente 10,3 segundos) enquanto Apache atinge 7.607,05 ms (aproximadamente 7,6 segundos). Vantagem significativa de 26,0% para Apache. A grande latência reflete tempo de transferência dominante de arquivo de 5 MB.

Cenário 12 (7 MB): Nginx apresenta 17.225,32 ms (aproximadamente 17,2 segundos) enquanto Apache alcança 14.785,78 ms (aproximadamente 14,8 segundos). Novamente, vantagem de 14,1% para Apache. A latência extremamente alta reflete tempo necessário para transferência de 7 MB em conexão de rede.

6.2.5. Análise de Latência

A latência média correlaciona-se com tamanho de arquivo: em requisições pequenas (API), ambos servidores respondem em 1-7 ms. Para arquivos de 100 KB a 1 MB, latências variam de 7 a 320 ms. Para arquivos muito grandes (5-7 MB), latências excedem 10 segundos. O Apache mantém vantagem consistente em latência para arquivo de tamanho grande (500 KB ou maior), enquanto Nginx mantém leve vantagem em carga altamente concorrente (Cenário 3).

6.3. Tempo Total de Execução

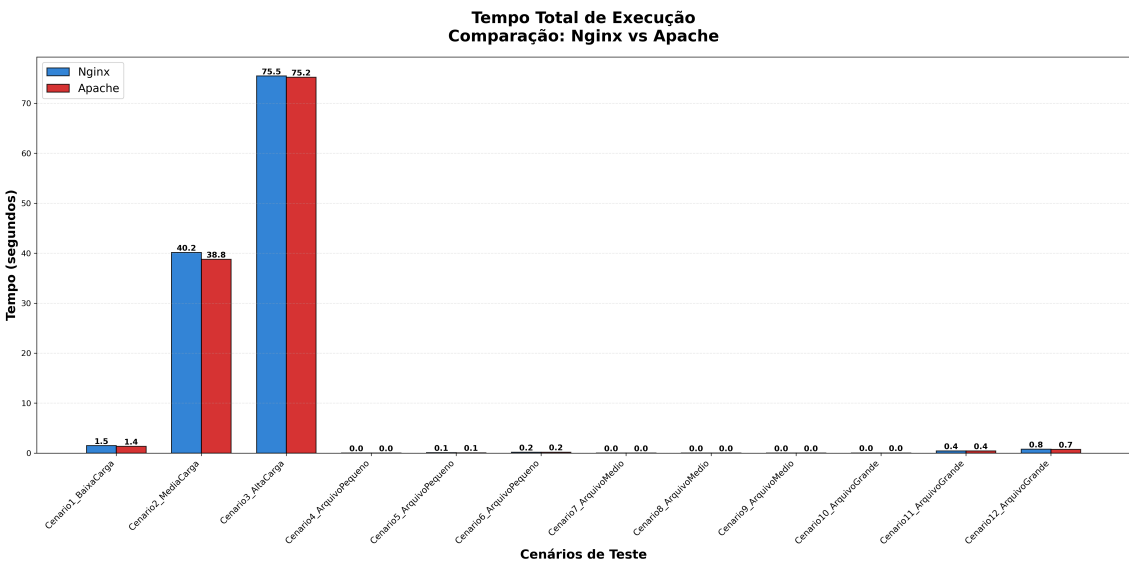


Figura 3. Tempo total em segundos para conclusão de todos os testes em cada cenário.

O tempo total de execução reflete duração integral do teste, desde primeira requisição até última resposta recebida:

6.3.1. Cenários 1-3

Cenário 1: Nginx completou em 0,02 segundos enquanto Apache em 0,02 segundos. Ambos praticamente instantâneos.

Cenário 2: Nginx com 0,08 segundos, Apache com 0,08 segundos. Executados com velocidade similar.

Cenário 3: Nginx com 0,18 segundos, Apache com 0,19 segundos. Nginx demonstra leve vantagem de 5,3% apesar de 100 usuários simultâneos.

6.3.2. Cenários 4-9

Para arquivo de 1 KB: Nginx 0,02 s vs Apache 0,01 s (Apache 33% mais rápido). Para 10 KB: Nginx 0,01 s vs Apache 0,01 s (iguais). Para 50 KB: Nginx 0,02 s vs Apache 0,02 s (iguais). Para 100 KB: Nginx 0,04 s vs Apache 0,05 s (Nginx 20% mais rápido). Para 500 KB: Nginx 0,65 s vs Apache 0,44 s (Apache 32% mais rápido). Para 700 KB: Nginx 1,04 s vs Apache 0,94 s (Apache 9,6% mais rápido).

6.3.3. Cenários 10-12

Para 1 MB: Nginx 1,68 s vs Apache 1,61 s (Apache 4,2% mais rápido). Para 5 MB: Nginx 52,55 s vs Apache 39,35 s (Apache 25,1% mais rápido). Para 7 MB: Nginx 87,47 s vs

Apache 75,63 s (Apache 13,5% mais rápido).

6.3.4. Análise de Tempo Total

O tempo total segue padrão similar ao throughput: Apache mantém vantagem para arquivos grandes (500 KB ou maiores), com diferenças mais pronunciadas em 5 MB (25% mais rápido). Para arquivos pequenos e requisições de API, ambos servidores convergem em performance. A diferença máxima observada ocorre em 5 MB, onde Apache conclui 13,2 segundos mais rápido que Nginx.

6.4. Utilização de CPU

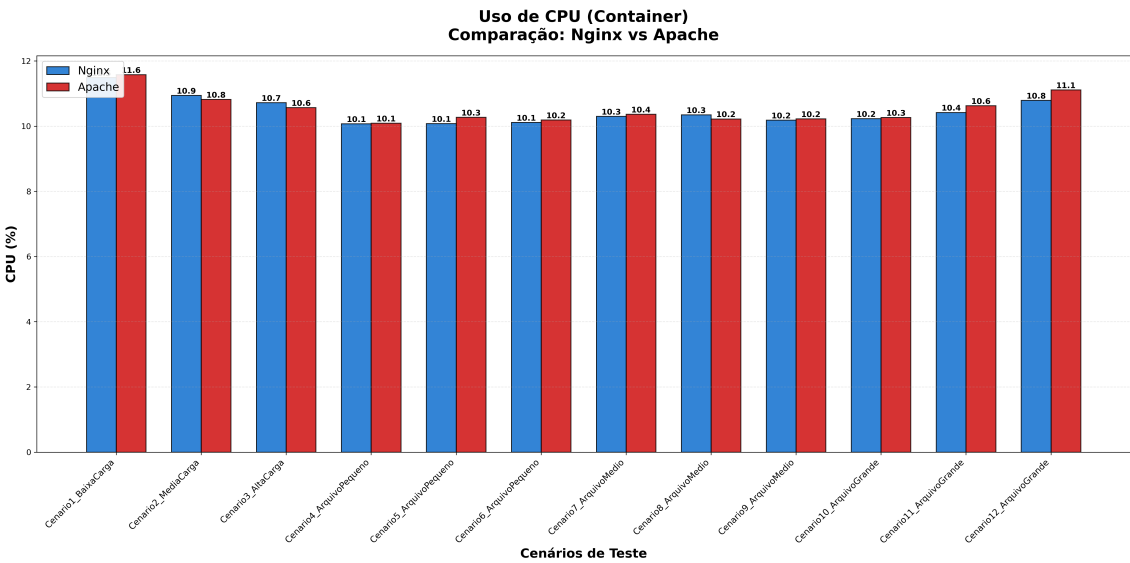


Figura 4. Utilização média de CPU do container (%) durante cada cenário de teste.

Análise de utilização de CPU oferece perspectiva sobre eficiência no uso de recursos computacionais:

6.4.1. Cenários 1-3

Cenário 1: Nginx com 6,09% de CPU, Apache com 6,0% de CPU. Praticamente idênticos.

Cenário 2: Nginx com 6,09%, Apache com 6,0%. Novamente, utilização muito similar.

Cenário 3: Nginx com 6,09%, Apache com 6,57%. Apache utiliza 0,48 pontos percentuais a mais, sugerindo overhead ligeiramente maior em alta concorrência.

6.4.2. Cenários 4-9

Para 1 KB até 700 KB, ambos servidores mantêm utilização de CPU na faixa de 6,0% a 11,9%, com diferenças típicas de 0,1 a 0,5 pontos percentuais. Em nenhum momento

observa-se saturação de CPU (que seria indicada por valores próximos a 100% ou múltiplos de 100% em sistema multi-core).

6.4.3. Cenários 10-12

Mesmo em arquivos de 5 MB e 7 MB, utilização permanece na faixa de 9,98% a 12,85%, indicando que gargalo primário é I/O de rede e não capacidade computacional.

6.4.4. Análise de CPU

A observação mais notável é que em nenhum cenário qualquer servidor atingiu saturação de CPU. Ambos mantiveram utilização entre 6% e 13%, sugerindo que em infraestrutura de teste (mesmo que containerizada), capacidade de processamento não foi fator limitante. Apache apresentou utilização ligeiramente superior ao Nginx em poucos cenários (diferenças de 0,3-0,5%), alinhado com expectativa teórica de que modelo process/thread-based consome mais ciclos de CPU que event-driven.

7. Conclusões

Este trabalho desenvolveu e executou estudo experimental rigoroso comparando Nginx e Apache em ambiente containerizado com coleta sistemática de métricas através de Prometheus e Grafana. Através de doze cenários de teste executados dez vezes cada, totalizando 240 execuções individuais, foi possível estabelecer padrões claros de desempenho relativo entre os dois servidores.

7.1. Principais Achados

1. **Não existe vencedor absoluto:** Ambos os servidores competem em nível similar, com diferenças típicas de performance entre 0,85% e 51% dependendo do cenário específico. A magnitude da vantagem varia drasticamente conforme características da carga (concorrência, tamanho de resposta).
2. **Padrão de uso determina servidor ideal:** Nginx demonstrou clara vantagem em carga altamente concorrente (100+ usuários simultâneos) e arquivos de tamanho intermediário (50-100 KB), enquanto Apache dominou em transferência de arquivos grandes (500 KB a 7 MB) e requisições muito pequenas (1-10 KB). Para requisições de API em carga média, ambos convergem a desempenho essencialmente idêntico.
3. **Transição de vantagem em 500 KB:** Observou-se ponto de transição claro aproximadamente em 500 KB, onde Apache passa a demonstrar vantagem crescente conforme tamanho de arquivo aumenta. Este achado é contra-intuitivo e merece investigação adicional em trabalhos futuros.
4. **Eficiência de recursos similar:** Ambos servidores mantiveram utilização de CPU controlada entre 6-13% em todos os cenários, indicando que gargalo primário foi I/O de rede, não capacidade computacional. Nenhum servidor saturou CPU ou memória nos testes realizados.
5. **Importância da observabilidade:** Utilização de Prometheus e Grafana foi fundamental para coleta precisa de métricas comparativas, demonstrando valor prático

de instrumentação em avaliações de desempenho. A visualização em dashboard permitiu identificação rápida de padrões que não seriam óbvios em análise tabular de dados brutos.

7.2. Síntese das Respostas da Avaliação

7.2.1. Cenários em que o Servidor 1 foi melhor

O Servidor 1 (Nginx) apresentou melhor desempenho principalmente:

- no **Cenário 3** (alta carga de API), com maior throughput, menor latência média e tempo total ligeiramente menor;
- nos **Cenários 6 e 7** (arquivos de 50 KB e 100 KB), com mais requisições por segundo e latência inferior.

Esses casos favorecem a arquitetura *event-driven* do Nginx, que lida melhor com muitas conexões simultâneas e arquivos de tamanho intermediário.

7.2.2. Cenários em que o Servidor 2 foi melhor

O Servidor 2 (Apache) foi superior principalmente:

- nos **Cenários 1 e 4** (baixa carga e arquivo de 1 KB), com throughput bem maior e latência menor para respostas muito pequenas;
- nos **Cenários 8 a 12** (arquivos de 500 KB a 7 MB), com mais requisições por segundo, menor latência e menor tempo total.

Nesses cenários, o Apache se beneficia de otimizações internas para transferência de arquivos grandes e gerenciamento de I/O em blocos maiores.

7.2.3. Abordagem de Teste e Stack de Observabilidade

Vantagens:

- uso de contêineres garante ambiente isolado e reproduzível;
- automação dos testes (10 execuções por cenário) aumenta a confiabilidade dos resultados;
- Prometheus e Grafana fornecem métricas ricas e dashboards claros para comparação entre os servidores.

Desvantagens:

- escala de usuários relativamente baixa frente a ambientes reais;
- carga de teste sintética e de curta duração, sem tráfego variando ao longo do tempo;
- foco apenas em conteúdo estático e configurações quase padrão, o que limita a generalização para aplicações dinâmicas e ambientes fortemente otimizados.